

Release-Candidate Dev Cycle — "Vyomi built on Vyomi"

The dev workflow Vyomi commits to going forward:

```
Mac (developer laptop)
├─ multipass VM: cloudlearn-appliance ← your daily-driver Vyomi
│   (the appliance you use day-to-day; you upgrade it ONLY after
│   a release is signed off – no live-edit on production)
└─ multipass VM: vyomi-ec2-rc          ← the RC validation rig
    (every Release Candidate is deployed here BEFORE promotion;
    this is where you exercise an RC before it becomes a release)
```

All dev validation happens on **Release Candidates**, deployed onto the dev rig via CI/CD, validated by the dev, and only then promoted to a final release. The Mac appliance is upgraded only after promotion, and development continues against the promoted release.

The cycle in 7 steps

```
1. Local code changes
   (edits to server.py, core/, static/, etc.)

↓

2. Cut an RC tag
   git tag v2.0.4-rc1 && git push --tags

↓

3. CI fires release-candidate.yml
   • Builds tarball + .deb + .rpm + SHA256SUMS
   • Builds docker image vyomi/appliance:2.0.4-rc1
   • Publishes GitHub Pre-release
   • Does NOT bump brew/scoop/snap downstream

↓

4. Deploy to dev rig
   vyomi deploy-rc v2.0.4-rc1
   • Pulls tarball from GitHub Pre-release
   • Pushes to vyomi-ec2-rc
```

- Builds + starts docker compose stack
- Reports URL: `http://192.168.x.y:9000/`

|



5. Validate

Open the URL, test the SPA, run conformance.
Check `vyomi rc-status` for compose health.
This is the gate – if anything looks wrong,
cut a `v2.0.4-rc2` with the fix and goto 4.

|



6. Promote

`vyomi promote-rc v2.0.4-rc1`

- Re-tags the RC commit as `v2.0.4` (no `-rc1`)
- `git push` fires `release.yml`
- Brew tap + Scoop bucket bump
- Docker image gets `:2.0.4` + `:latest` tags

|



7. Upgrade your Mac appliance

`brew upgrade vyomi && vyomi restart`
Your daily driver now runs `v2.0.4`.
Continue development → step 1 for `v2.0.5`

One-time setup

```
# Install the dev rig (4 vCPU / 8GB / 30GB multipass VM with docker)
vyomi rc-init

# That's it. The CI workflows release-candidate.yml + release.yml are
# already wired in this repo.
```

CLI reference

Command	What it does
<code>vyomi rc-init</code>	Creates the <code>vyomi-ec2-rc</code> multipass VM, installs docker + compose, pre-pulls base images. Idempotent — safe to re-run.

Command	What it does
<code>vyomi deploy-rc <tag></code>	Downloads the tarball for an RC from GitHub, pushes to <code>vyomi-ec2-rc</code> , brings up the compose stack, waits for <code>/healthz</code> .
<code>vyomi rc-status</code>	Shows what RC is deployed, VM state, compose service health, the dev validation URL.
<code>vyomi promote-rc <rc-tag></code>	Promotes an RC to a final release. Strips <code>-rcN</code> / <code>-betaN</code> / <code>-alphaN</code> suffix, re-tags the commit, pushes. Safety checks: RC must exist as Pre-release, final tag must not exist, deployment evidence on <code>vyomi-ec2-rc</code> must be present (override with <code>--force</code>).

vyomi deploy-rc flags

Flag	Effect
<code>--local <path></code>	Use a local <code>.tar.gz</code> instead of downloading from GitHub. Useful for testing changes before they're cut as a tag.
<code>--no-build</code>	Pull <code>vyomi/appliance:<tag></code> from Docker Hub instead of building from tarball. Faster but requires the <code>docker-publish.yml</code> workflow to have completed.
<code>--vm <name></code>	Override target VM (default: <code>vyomi-ec2-rc</code>).
<code>--port <n></code>	Override host port (default: <code>9000</code>).
<code>--skip-restart</code>	Stage the source on the VM but don't bring the stack up.
<code>-v, --verbose</code>	Stream full docker build + compose output.

vyomi promote-rc flags

Flag	Effect
<code>--force</code>	Skip the "deployed onto vyomi-ec2-rc" safety check. Reserved for emergency promotions where the dev rig is unavailable.
<code>--dry-run</code>	Show the exact <code>git tag</code> + <code>git push</code> commands without executing.

Why this shape

Choice	Alternative	Reason
Multipass VM (not LXD container)	LXD container	Vyomi-EC2 dev rig must host docker compose; LXD containers don't nest docker reliably.
Separate dev rig (vyomi-ec2-rc)	Run RC on the same <code>cloudlearn-appliance</code>	A broken RC on the daily-driver costs you work. Separate rig keeps the cycle safe.
<code>vyomi deploy-rc</code> is manual, not auto	Polling daemon / push webhook	<code>cloudlearn-appliance</code> is behind NAT; GitHub Actions can't push. Polling adds operational surface. Manual = intentional = predictable.
RC artifact is a tarball + docker build	Direct docker image pull	Building from tarball matches what real customer installs do via brew formula. Validates the full build chain.
Promote-RC uses <code>git tag</code> on the RC commit	Re-build from a new commit	Keeps the artifact byte-identical between RC and final. What you validated is exactly what ships.
Dev appliance upgrade is <code>brew upgrade</code>	Auto-upgrade daemon	Same release path real customers use. Validates the upgrade UX on every release.

Failure modes + recovery

`vyomi deploy-rc` fails with `/healthz unreachable`

```
# Inspect the running compose stack on the dev rig
multipass exec vyomi-ec2-rc -- sudo docker compose \
  -f /home/ubuntu/cloud-learn-<tag>/docker-compose.appliance.yml ps

# Tail the simulator logs
multipass exec vyomi-ec2-rc -- sudo docker logs cloud-learn-simulator-1 --tail 50

# Tail the deploy-rc compose log
multipass exec vyomi-ec2-rc -- tail -100 /home/ubuntu/cloud-learn-<tag>-compose.log
```

Don't promote this RC. Cut a new `-rc` tag with the fix.

`vyomi promote-rc` rejects with "deployment evidence not found"

The RC was tagged + pushed but `vyomi deploy-rc` was never run against it. Either:

```
vyomi deploy-rc v2.0.4-rc1      # Deploy + validate first, then re-run promote-rc
# OR
vyomi promote-rc v2.0.4-rc1 --force  # Override (rare; use for emergency fixes)
```

`vyomi-ec2-rc` multipass VM is corrupted / unrecoverable

```
multipass delete vyomi-ec2-rc --purge
vyomi rc-init                          # Fresh VM
```

This loses any RC-state on the rig (acceptable — it's a validation rig, not production).

CI release-candidate.yml fails

Check the run log at <https://github.com/vyomi-cloud/appliance/actions>. Common issues:

- **Tarball builds but RPM fpm fails** — usually a dep missing from packaging.
- **github-release fails** — usually a CHANGELOG.md formatting issue.

Re-run via `gh workflow run release-candidate.yml -F version=2.0.4-rc1` after fixing.

Conformance gates (must be green before promote-rc)

Two test suites guard the release:

`tests/conformance/ui/` — API-level (fast, ~3 min)

Catalog-schema-driven HTTP tests using `requests`. Asserts:

- The SPA wizard's payload shape matches the backend's Pydantic model
- Catalog endpoints accept the SPA-shaped payload
- Backend writes the resource to STATE
- List endpoint returns the resource in JSON

Catches: field-name mismatches (Pydantic aliases), `tags`-as-dict acceptance, RDS routing through `gcp_sql_engine`, etc.

Does NOT catch: SPA rendering bugs (the JSON contract can be green while the user sees a blank page).

`tests/conformance/ui_real/` — real-browser SPA (slower, ~5-10 min)

Playwright tests that actually drive Chromium. For every catalog service:

```
playwright.goto(/console/<provider>)
→ click rail item
→ click Create
→ walk every wizard tab (fill visible fields, click Next)
→ click submit on last tab
→ wait for SPA navigate→list
→ ASSERT a DOM row containing the new identifier is visible
→ ASSERT no error toast fired
```

Catches: envelope parser misses (the `db_instances` bug shipped in v2.0.0-v2.0.3 would have failed this), auto-refresh wiping selections, disabled-button silent failures, any future JS regression in the wizard submit

pipeline.

Failures dump screenshot + browser console + `/api/` network log to `/tmp/vyomi-ui-real-failures/` for triage.

Wiring in the cycle

```
git tag v2.0.4-rc1 && git push --tags
  ↓ CI release-candidate.yml builds the artifact

vyomi deploy-rc v2.0.4-rc1
  ↓ deploys to vyomi-ec2-rc

# RUN BOTH conformance suites before promoting:
.venv/bin/pytest tests/conformance/ui/      # ~3 min, must be green
.venv/bin/pytest tests/conformance/ui_real/  # ~10 min, must be green

vyomi promote-rc v2.0.4-rc1
  ↓ re-tags as v2.0.4, fires release.yml
```

What's NOT in scope yet (roadmap)

- **Auto-upgrade daemon on dev rig** — currently you run `vyomi deploy-rc` manually after every CI completion. A future tweak would poll GitHub releases and auto-deploy new RCs.
- **Vyomi-managed EC2 API integration** — currently `vyomi-ec2-rc` is a raw multipass VM that doesn't show up in `vyomi`'s AWS EC2 API. v2.1.0+ refactor will let the simulator's `/api/ec2/instances` create + manage multipass-backed instances.
- **vyomi rc-conformance** — single command that runs BOTH conformance suites against the deployed RC and gates `vyomi promote-rc` on the result. Scaffolding is here today (you run them separately); the command wrapper is the v2.0.5 cleanup.